

Examen final : Compléments d'informatique

ENSAI 2A UE3

2026-09-28

Instructions

- Examen écrit
- Durée : 2 heures
- Une feuille A4 de notes **manuscrites** (recto et verso) est autorisée
- Calculatrice interdite

! À lire avant de commencer

- Vous pouvez répondre aux exercices dans l'ordre de votre choix.
- Au sein d'un même exercice, les questions ne sont pas nécessairement classées par difficulté croissante.
- Sauf indication contraire, les questions attendent des réponses courtes.
- Suivez strictement les consignes. Si une question vous semble ambiguë, indiquez vos hypothèses sur votre copie.
- Lorsque du code est demandé, respectez les conventions de base du langage (mots-clés, indentation, etc.). Les erreurs de syntaxe dues à l'écriture manuscrite seront tolérées. Si vous ne vous souvenez pas du nom exact d'une méthode Python, vous pouvez utiliser du pseudocode.
- Sauf demande explicite, la documentation du code n'est pas requise.

Bon courage !

1 Questions de cours (3 points)

- a. Donnez six exemples de fichiers ou dossiers couramment présents dans un dépôt de projet informatique.
- b. Listez les principaux acteurs impliqués dans un projet informatique.
- c. Qu'est-ce qu'un design pattern ?
- d. Quelles méthodes HTTP correspondent aux opérations CRUD ?
- e. Écrivez une requête HTTP permettant de créer un nouveau Pokemon (nom : Ponyta, type : Fire, niveau : 12) dans <https://pokedex-api> (choisissez un endpoint approprié).

2 Questions à choix multiple (8 points)

Chaque question peut comporter une ou plusieurs réponses correctes (au moins une réponse est correcte). Réponse correcte : 1 point. Réponse partiellement correcte : au prorata. En cas de réponse incorrecte : 0.

2.1 Development tools : quelles affirmations sont correctes ?

- a. `pytest` est utilisé pour exécuter des test unitaires.
- b. `psycopg2` est utilisé pour communiquer avec PostgreSQL.
- c. `requests` est utilisé pour communiquer avec PostgreSQL.
- d. `FastAPI` est principalement utilisé pour formater du code source Python.
- e. Le module `abc` peut être utilisé pour définir des classes abstraites.
- f. Swagger peut être utilisé pour explorer et tester une API.

2.2 Pourquoi logging peut-il être préférable à l'utilisation d'instructions `print()` lors du debugging d'une application ?

- a. Les logs peuvent indiquer le niveau de gravité d'un message.
- b. Les logs peuvent être configurés pour être écrits dans un fichier.
- c. Les logs peuvent fournir des informations sur le moment où un événement s'est produit.
- d. Les instructions `print()` ne peuvent afficher aucune information sur une erreur.
- e. Logging est uniquement utile lorsqu'une application plante.

2.3 Quelles affirmations décrivent de bonnes pratiques pour les test unitaires ?

- a. Un test unitaire doit vérifier un comportement précis.
- b. Les tests doivent être reproductibles.
- c. Un test devrait idéalement être indépendant des systèmes externes.
- d. Un test unitaire doit toujours accéder à la base de données de production réelle.
- e. Les tests peuvent aider à détecter les régressions après une modification du code.
- f. Un test unitaire dépend d'un ordre d'exécution spécifique.

2.4 Quelles affirmations concernant les mocks sont correctes ?

- a. Un mock teste toujours l'implémentation réelle du composant qu'il remplace.
- b. Il peut être utilisé pour isoler le code testé de ses dépendances externes.
- c. Un mock permet à un test de simuler des comportements spécifiques d'une dépendance.
- d. Il est uniquement utile lors du test de code qui accède à une base de données.
- e. Un mock est un objet simulé qui remplace un composant réel pendant un test.

2.5 Un utilisateur soumet un formulaire pour créer un nouveau player. Quelle séquence représente le mieux le flux d'une requête à travers les couches de l'application ?

- a. DAO → Controller → Service → Database
- b. Controller → DAO → Service → Database
- c. Service → Client → DAO → Database
- d. Controller → Service → DAO → Database
- e. Service → Controller → DAO → Database

2.6 Quelles affirmations décrivent les avantages d'une application layered bien conçue ?

- a. Les Controllers devraient contenir la majeure partie de la logique métier.
- b. Chaque couche a une responsabilité spécifique.
- c. Si le database management system (DBMS) change, l'impact peut être plus limité car le code spécifique à la base de données est principalement isolé dans la layer DAO.
- d. N'importe quelle couche peut accéder librement à la base de données et la modifier.
- e. La séparation des responsabilités peut rendre l'application plus facile à lire, debugger et maintenir.
- f. La logique métier est séparée de l'accès aux données et des interactions avec l'utilisateur.

2.7 Un utilisateur clique sur un bouton de l'UI (User interface) pour récupérer ses games. Quelles affirmations sont correctes ?

- a. La base de données reçoit directement le clic de l'utilisateur depuis l'UI.
- b. Le frontend envoie une requête HTTP à la backend API.
- c. Le backend traite la requête et demande à la base de données les données nécessaires.
- d. La base de données peut renvoyer des données au backend, qui envoie ensuite une réponse au frontend.
- e. L'UI doit toujours être implémentée dans le backend.
- f. L'API peut être utilisée comme interface entre le frontend et le backend.

2.8 Quelles requêtes HTTP sont appropriées pour récupérer toutes les games jouées par le player 96 ?

- a. PUT `https://my-awesome-url/game?id_player=96`
- b. GET `https://my-awesome-url/game?id_player=96`
- c. PUT `https://my-awesome-url/game/96`
- d. GET `https://my-awesome-url/player/96`
- e. GET `https://my-awesome-url/player/69/games`
- f. GET `https://my-awesome-url/player/96/games`

2.9 Quelles affirmations sont correctes concernant les environment variables ?

- a. Les variables d'environnement peuvent être utilisées pour configurer la connexion à la base de données.
- b. Un mot de passe peut être stocké dans une variable d'environnement.
- c. Le fichier `.env` contenant des secrets ne doit pas être commité dans le remote repository.
- d. Les identifiants de connexion à la base de données devraient de préférence être écrits en dur dans chaque DAO.
- e. L'utilisation de variables de configuration facilite le changement d'environnement.
- f. L'utilisation de variables d'environnement signifie que les secrets ne peuvent pas être accessibles par l'application.

3 Code (5 points)

Ci-dessous se trouvent deux classes représentant des business objects dans une application de gaming.

```
from business_object.player import Player

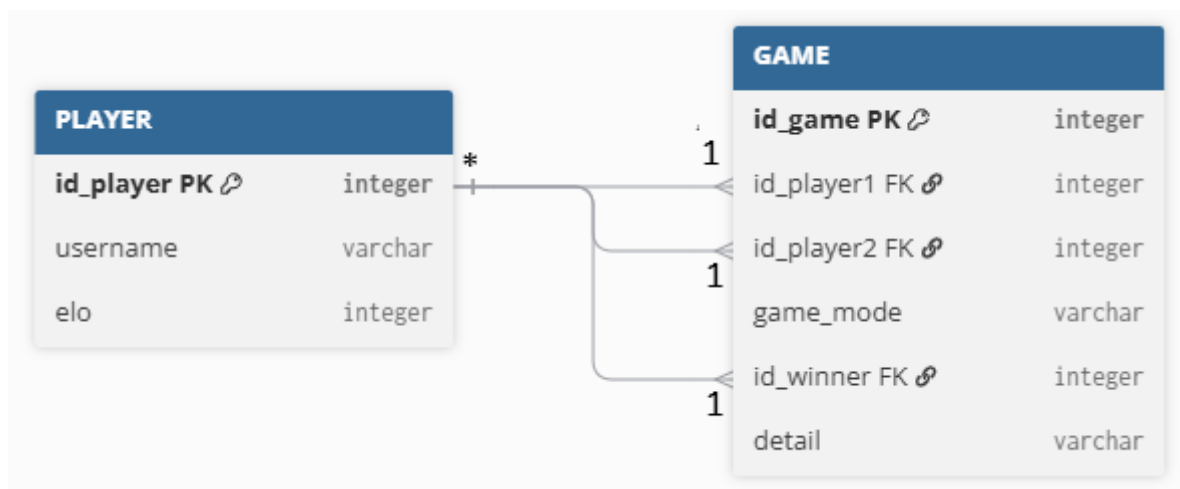
class Game:
    """Business object representing a Game between two players."""
    def __init__(self, player1: Player, player2: Player, game_mode: str,
                  winner: Player | None, id_game: int=None):
        self.id_game = id_game
        self.player1 = player1
        self.player2 = player2
        self.game_mode = game_mode
        self.winner = winner
```

```
class Player:
    """Business object representing a Player."""
    def __init__(self, username: str, elo: int, pokemon_fan:bool=False, id_player=None):
        self.id_player = id_player
        self.username = username
        self.elo = elo
        self.pokemon_fan = pokemon_fan
```

- Créez un Game de type coin flip entre Fred et Jamy.
- Créez une nouvelle méthode qui renvoie une représentation sous forme de chaîne de caractères du Game, telle que :

“coinflip between Fred and Jamy. Winner: Jamy”

Ces données sont stockées selon ce modèle :



- Écrivez une requête SQL pour lister toutes les games de type dice remportées par le premier player (*player1*), classées selon la primary key de la game.
- Complétez la fin de la méthode `find_by_id()` ci-dessous :

```

class GameDao(Singleton):
    def find_by_id(self, id_game: int) -> Game | None:
        """
        Retrieves a specific game and converts the database row into a Game object.
        Args:
            id_game (int): The ID of the game to find.
        Returns:
            Game: The game object if found, otherwise None."""
        res = None
        with DBConnection().connection as connection:
            with connection.cursor() as cursor:
                cursor.execute(
                    """
                    SELECT *
                    FROM game
                    WHERE id_game = %(id_game)s;
                    """,
                    {"id_game": id_game},
                )
                res = cursor.fetchone()
        if not res:
            return None
        p1 = PlayerDao().find_by_id(res["id_player1"])

```

e. En suivant la documentation, écrivez le code de cette méthode :

```

class GameModeFactory:
    @classmethod
    def get_mode(cls, game_mode: str) -> GameMode:
        """
        Returns the corresponding GameMode object.
        Args:
            game_mode (str): The identifier of the game mode (e.g., 'coinflip', 'dice').
        Returns:
            CoinFlipMode or DiceMode: An instance of a class implementing GameMode.
        Raises:
            ValueError: If the requested game_mode is not supported."""

```

Voici un test unitaire qui vérifie qu'une exception est levée.

```

def test_unknown_mode():
    with pytest.raises(<?>):
        GameModeFactory.get_mode(<?>)

```

f. Complétez ce test et écrivez-en deux autres pour tester la méthode `get_mode()`.

Astuce : `isinstance(<object>, <class_or_type>) -> bool`

4 Git (4 points)

Sofia vient de rejoindre votre équipe de développement en tant que junior developer.

Votre équipe travaille sur une application professionnelle, hébergée dans le Git repository suivant : <https://github.com/goodfood/ten-vegetables-per-day.git>

Sofia n'a jamais travaillé avec Git auparavant et doit travailler pour la première fois sur le projet. Vous décidez de lui expliquer le workflow de base.

- Quelle commande Git doit-elle utiliser pour récupérer une copie locale du repository ?
- Quel est l'objectif de l'utilisation d'un personal access token lors de l'authentification à GitHub via HTTPS ?
- Décrivez le Git workflow que vous utilisez chaque semaine pour mettre à jour le fichier partagé du projet de suivi de projet.

Si vous ne travaillez pas sur le projet : indiquez-le et imaginez que chaque semaine vous devez remplir un fichier contenant une liste de tâches et l'envoyer vers le dépôt distant.

Sofia modifie `src/vegetables.py` et essaie de push ses modifications, mais Git affiche « Everything up-to-date » / « Nothing to push ».

- Que doit-elle faire avant de push ses modifications ?

Sofia essaie à nouveau de push son commit, mais cette fois Git affiche un message différent.

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/goodfood/ten-vegetables-per-day.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes.
```

- Pourquoi Sofia obtient-elle ce message ? Que doit-elle faire pour résoudre la situation ?

Cependant, Sofia est en réalité une espionne. Sa mission est de faire disparaître l'application **Ten Vegetables Per Day**.

Pendant une pause déjeuner, tous les membres de l'équipe laissent leurs ordinateurs déverrouillés. Sofia en profite pour supprimer toutes les copies locales du projet sur les ordinateurs des développeurs.

Elle trouve également un Post-it contenant le mot de passe du compte GitHub et l'utilise pour supprimer le remote repository.

Sofia pense avoir réussi. Cependant, elle n'a pas remarqué que Stan est en télétravail ce jour-là. Son ordinateur n'est pas connecté au réseau local et n'a pas été affecté.

- Le projet est-il nécessairement perdu ? Expliquez pourquoi.